



9. Aprendendo a utilizar dicionários em Python

☒ Dúvidas?	☐
☒ Exercícios a entregar?	☒
 Módulo	1 - Primeiros passos com Python e Versionamento de código
 Tipo de aula	Teste Vídeo
 Fora do prazo	 Fora do prazo
 Links e mídias	Pasta "Aula 9"
 Última atualização	@17 de novembro de 2025 04:00
 Status	Done
 Data início	@12/11/2025

📅 Data conclusão	@14/11/2025 0:17
⌚ Esforço	Médio

Anotações

Vídeo 1 - Dicionários: criação e acesso aos dados

Acesso aos dados

Dicionários aninhados

Vídeo 2 - Métodos da classe `dict`

```
{}.clear
{}.copy
{}.fromkeys
{}.get
{}.items
{}.keys
{}.pop
{}.popitems
{}.setdefault
{}.update
{}.values
in
del
```

Exercícios

Arquivos

Anotações

Observação: esta lição ficou "Fora do prazo" porque tive que ir para a casa da Solanger ajudar com a questão da elétrica.

Vídeo 1 - Dicionários: criação e acesso aos dados

Um dicionário é um conjunto não ordenado de pares chave:valor, em que as chaves são únicas em uma dada instância do dicionário. Para delimitar (declarar) um dicionário, usamos chaves (`{ }`) contendo uma lista de pares chave:valor separados por vírgulas (`,`), ou o construtor `dict` . Note que, ao usar o construtor, **não se coloca** aspas para as chaves dentro dos parênteses.

Para que a chave de um dicionário seja válida, ela precisa ter um valor **imutável**. Porém, os valores podem ser **mutáveis** ou **imutáveis**. Então uma lista

não pode ser uma chave em um dicionário, isso faz com que a criação do dicionário seja impedida.

```
peessoa = {"nome": "lizandra", "idade": 44} # {'nome': 'lizandra', 'idade': 44}

peessoa1 = dict(nome="lizandra", idade=44) # {'nome': 'lizandra', 'idade': 44}

# Adicionar um novo par de chave/valor ao dicionário
peessoa["telefone"] = "99999-1234" # {'nome': 'lizandra', 'idade': 44, 'telefone': '99999-1234'}
peessoa1["telefone"] = "99999-1234" # {'nome': 'lizandra', 'idade': 44, 'telefone': '99999-1234'}
```

Acesso aos dados

Acessamos os dados de um dicionário através de suas chaves, **e não seus índices**. Para alterar os valores, também usamos as chaves, atribuindo o novo valor. Qualquer atribuição mencionando uma das chaves existentes **substitui** seu valor correspondente, sem exibi-lo, então é preciso cuidado para não alterar algo de forma indesejada.

```
dados = {'nome': 'lizandra', 'idade': 44, 'telefone': '99999-1234'}

dados["nome"] # lizandra
dados["idade"] # 44
dados["telefone"] # 99999-1234

dados["nome"] = "Maria"
dados["idade"] = 18
dados["telefone"] = "99988-5678"

dados # Maria, 18, 99988-5678
```

Dicionários aninhados

Os valores de dicionários podem armazenar qualquer tipo de objeto do Python, desde que sua chave respectiva seja um valor imutável.

```
contatos = {
    "guilherme@gmail.com": {"nome": "Guilherme", "telefone": "3333-2221"},
    "giovana@gmail.com": {"nome": "Giovana", "telefone": "3443-2121"},
    "chappie@gmail.com": {"nome": "Chappie", "telefone": "3344-9871"},
    "melaine@gmail.com": {"nome": "Melaine", "telefone": "3333-7766"}
}
```

`contatos["giovana@gmail.com"]["telefone"]` # 3443-2121, exibe o telefone do contato com esse email

Cada *e-mail* acima é uma chave, e cada chave contém como valor um novo dicionário, cada um com dois pares de chave/valor.

Este tipo de aninhamento pode ser usado como um banco de dados, já que permite inúmeros níveis aninhados.

Assim como listas e tuplas, é possível iterar um dicionário usando `for` ou `while`, porém **dê preferência** para usar o método `items` (veja abaixo) ao iterar dicionários, que retorna um conjunto de tuplas, em que o primeiro elemento é a chave e o segundo, o valor.

Vídeo 2 - Métodos da classe `dict`

`{}.clear`

Exclui todos os elementos de um dicionário de uma só vez.

```
contatos = {
    "guilherme@gmail.com": {"nome": "Guilherme", "telefone": "3333-2221"},
    "giovanna@gmail.com": {"nome": "Giovanna", "telefone": "3443-2121"},
    "chappie@gmail.com": {"nome": "Chappie", "telefone": "3344-9871"},
    "melaine@gmail.com": {"nome": "Melaine", "telefone": "3333-7766"},
}

contatos.clear()
print(contatos) # {}
```

`{}.copy`

Funciona de modo semelhante ao método `copy` das listas: criando uma cópia para que o dicionário original não seja alterado em outro ponto do código.

```
contatos = {"guilherme@gmail.com": {"nome": "Guilherme", "telefone": "3333-2221"}}

copia = contatos.copy()
copia["guilherme@gmail.com"] = {"nome": "Gui"}

print(contatos["guilherme@gmail.com"]) # {"nome": "Guilherme", "telefone": "3333-2221"}

print(copia["guilherme@gmail.com"]) # {"nome": "Gui"}
```

`{}.fromkeys`

Cria novas chaves em um dicionário. São dois os casos de uso:

- Para criar somente as chaves, sem vincular um valor a elas (cria as chaves com o valor `None`).
- Para criar um conjunto de chaves com um valor padronizado entre elas.

Vejamos exemplos:

```
# Caso 1
dict.fromkeys(["nome", "telefone"]) # {"nome": None, "telefone": None}

# Caso 2
dict.fromkeys(["nome", "telefone"], "vazio") # {"nome": "vazio", "telefone": "vazio"}
```

Esse método pode ser aplicado a dicionários existentes (para tanto, é só substituir o construtor `dict` no código acima pelo nome do dicionário desejado) ou a novos dicionários (usando `dict`, como acima).

`{}.get`

Um método muito utilizado.

É outra forma de acessar valores dentro de um dicionário, caso não se saiba o nome da chave, por exemplo.

Caso a chave pesquisada não exista, exibe `KeyError` como resultado.

```
contatos = {"guilherme@gmail.com": {"nome": "Guilherme", "telefone": "3333-2221"}}
```

```
contatos["chave"] # KeyError
```

```
resultado = contatos.get("chave") # None, padrão quando o método não encontra a chave solicitada
print(resultado)
```

```
resultado = contatos.get("chave", {}) # {}, devolve o valor inserido no segundo parâmetro
# esse valor não precisa ser um dicionário vazio, pode ser também uma string ou um número, por exemplo
print(resultado)
```

```
resultado = contatos.get(
    "guilherme@gmail.com", {}
) # {"guilherme@gmail.com": {"nome": "Guilherme", "telefone": "3333-2221"}}
# Como esta chave existe, ele não retorna o dicionário vazio, mas sim o resultado encontrado
print(resultado)
```

`{}.items`

Retorna uma lista de tuplas; muito útil quando se quer usar o dicionário em um iterável (`for` ou `while`).

```
contatos = {"guilherme@gmail.com": {"nome": "Guilherme", "telefone": "3333-2221"}}
```

```
contatos.items() # dict_items([('guilherme@gmail.com', {'nome': 'Guilherme', 'telefone': '3333-2221'})])
```

`{}.keys`

Retorna somente as chaves contidas no dicionário. Útil para quando se precisa conhecer as chaves de um dicionário.

```
contatos = {"guilherme@gmail.com": {"nome": "Guilherme", "telefone": "3333-2221"}}
```

```
contatos.keys() # dict_keys(['guilherme@gmail.com'])
```

`{}.pop`

Funciona de forma similar às listas: remove um valor do dicionário. Caso o valor informado como parâmetro não exista, é possível fornecer um segundo parâmetro (`"chave", {}`) para que o método retorne um dicionário vazio (similar a `get` acima).

```
contatos = {"guilherme@gmail.com": {"nome": "Guilherme", "telefone": "3333-2221"}}
```

```
contatos.pop("guilherme@gmail.com") # {'nome': 'Guilherme', 'telefone': '3333-2221'}
```

```
contatos.pop("guilherme@gmail.com", {}) # {}, esse valor não precisa ser um dicionário vazio, pode ser também uma string
```

`{}.popitem`

Muito similar à `pop`, mas não é preciso informar a chave como parâmetro, ele retira os itens sequencialmente, começando de trás para a frente. Caso não tenha mais nada a remover, exibe um `KeyError`.

```
contatos = {  
    "guilherme@gmail.com": {"nome": "Guilherme", "telefone": "3333-2221"},  
    "lizandra@gmail.com": {"nome": "lizandra", "telefone": "4444-3332"},  
}
```

```
print(contatos)
```

```
resultado1 = contatos.popitem() # ('guilherme@gmail.com', {'nome': 'Guilherme', 'telefone': '3333-2221'})
```

```
print(resultado1)

print(contatos)

resultado2 = contatos.popitem() # ("lizandra@gmail.com": {"nome": "lizandra", "telefone": "4444-3332"})
print(resultado2)
```

`{}.setdefault`

Também é um método muito utilizado. Recebe dois parâmetros: a chave a ser alterada e o valor padrão a ser inserido para a chave.

```
contatos = {
    "guilherme@gmail.com": {"nome": "Guilherme", "telefone": "3333-2221"}
}

contatos.setdefault("nome", "Giovana") # Guilherme
print(contatos["nome"]) # {"nome": "Guilherme", "telefone": "3333-2221"}

contatos.setdefault("idade", 28) # 28
print(contatos) {"nome": "Guilherme", "telefone": "3333-2221", "idade": 28}
```

Se o valor já existe no dicionário, o valor correspondente **não será alterado**.

É útil quando não se sabe se o par chave/valor existe ou não; com este método não precisamos criar condições (`if`) para verificar a existência ou não da chave, afinal ele só insere o par caso a chave não exista, caso contrário não altera nada.

`{}.update`

Possibilita atualizar o dicionário, sejam valores de chave ou inclusões de novas chaves.

```
contatos = {
    "guilherme@gmail.com": {"nome": "Guilherme", "telefone": "3333-2221"}
}
```



```
contatos.update({"guilherme@gmail.com": {"nome": "Gui"}})
print(contatos) # {"guilherme@gmail.com": {"nome": "Gui"}}
```

```
contatos.update({"giovana@gmail.com": {"nome": "Giovana", "telefone": "3
322-8181"}})
print(contatos) # {"guilherme@gmail.com": {"nome": "Gui"}, {"giovana@gm
ail.com": {"nome": "Giovana", "telefone": "3322-8181"}}
```

Veja que a chave `"guilherme@gmail.com"` teve seu dicionário interno alterado por completo, removendo o item `"telefone"`.

`{}.values`

Retorna somente os valores das chaves contidas no dicionário. Útil para quando não se precisa conhecer as chaves de um dicionário, mas precisa iterar sobre os valores.

```
contatos = {
    "guilherme@gmail.com": {"nome": "Guilherme", "telefone": "3333-222
1"},
    "giovanna@gmail.com": {"nome": "Giovanna", "telefone": "3443-2121"},
    "chappie@gmail.com": {"nome": "Chappie", "telefone": "3344-9871"},
    "melaine@gmail.com": {"nome": "Melaine", "telefone": "3333-7766"},
}

contatos.values()
# dict_values([{'nome': 'Guilherme', 'telefone': '3333-2221'}, {'nome': 'Giov
anna', 'telefone': '3443-2121'},
# {'nome': 'Chappie', 'telefone': '3344-9871'}, {'nome': 'Melaine', 'telefone':
'3333-7766'}])
```

`in`

É uma forma mais simples e elegante de conhecer as chaves de um dicionário.

```
contatos = {
    "guilherme@gmail.com": {"nome": "Guilherme", "telefone": "3333-222
1"},
    "giovanna@gmail.com": {"nome": "Giovanna", "telefone": "3443-2121"},
    "chappie@gmail.com": {"nome": "Chappie", "telefone": "3344-9871"},
}
```

```
"melaine@gmail.com": {"nome": "Melaine", "telefone": "3333-7766"},
}
```

```
# Sem o `in`
```

```
lista_chaves: list = contatos.keys()
```

```
has_reference = lista_chaves.count("guilherme@gmail.com")
```

```
OU
```

```
has_reference = lista_chaves.index("guilherme@gmail.com")
```

```
# para saber se a chave está no dicionário
```

```
# e depois criar o código do `if` a partir dessas variáveis, porém...
```

```
# Com o `in`
```

```
"guilherme@gmail.com" in contatos # True
```

```
OU
```

```
"idade" in contatos["guilherme@gmail.com"] # False
```

```
# é bem mais fácil
```

del

Outra forma, mais simples, de excluir chaves e valores de um dicionário.

```
contatos = {
```

```
    "guilherme@gmail.com": {"nome": "Guilherme", "telefone": "3333-2221"},
```

```
    "giovanna@gmail.com": {"nome": "Giovanna", "telefone": "3443-2121"},
```

```
    "chappie@gmail.com": {"nome": "Chappie", "telefone": "3344-9871"},
```

```
    "melaine@gmail.com": {"nome": "Melaine", "telefone": "3333-7766"},
```

```
}
```

```
del contatos["guilherme@gmail.com"]["telefone"] # exclui somente o telefone desta chave
```

```
del contatos["chappie@gmail.com"] # exclui toda a chave e seus valores
```

```
# del contatos # isso apaga todo o dicionário
```

```
# {'guilherme@gmail.com': {'nome': 'Guilherme'},
```

```
# 'giovanna@gmail.com': {'nome': 'Giovanna', 'telefone': '3443-2121'},
```

```
# 'melaine@gmail.com': {'nome': 'Melaine', 'telefone': '3333-7766'}}
```

```
print(contatos)
```

Exercícios

Primeira tentativa ⇒ 50% - 2/4 corretas **PRECISO REVISAR**

Após a revisão: 100% ⇒ 4/4 corretas

Arquivos

Pasta "Aula 9"